Tecnun
Universidad
de Navarra

ESCUELA DE INGENIERÍA
INGENIARITZA ESKOLA
SCHOOL OF ENGINEERING

C++ Laboratory Lessons

Lesson Nº 11

C++

PARA INGENIEROS

Computer Programming II

Dr. Paul Bustamante / Dr. Jesús Paredes

OUTLINE

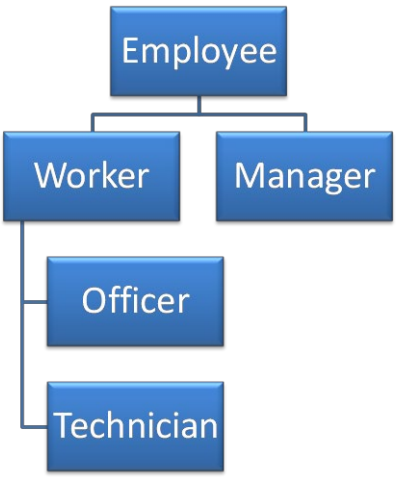
Contents

1. Introduction

This week's lesson is oriented to **Polymorphism**, for which you will create classes using inheritance and **virtual functions**. To start with, **what is polymorphism?** Polymorphism allows the name of a function to invoke a response on a base-class-type object and a different one in objects of a derived class, for which **virtual functions** are used.

1.1. Exercise 1:

This exercise deals with using ***virtual functions***, seeing how they act according to the object on which it is applied, which you will see in ***main()***.




```
class Employee
{
protected:
        double sal;          //salary base
public:
        Employee(double s){ sal=s;}
        double Payment(){ return sal;}
        void prt(){
            cout << "Salary="<< Payment() <<endl;
        }
};

class Manager : public Employee
{
        double inc;
public:
        Manager(double s, double i) : Employee(s) { inc = i; }
        double Payment(){ return sal*inc; }
};

void main()
{
        Employee e1(1500), e;
        Manager m1(1500, 1.5);
        cout << "Exercise about inheritance and polymorphism"<<endl;
        cout << "Salary: " << e1.Payment() << endl;
        cout << "Salary: " << m1.Payment() << endl;
```

```
        e = &e1;
        cout << "Salary: " << e->Payment() << endl;
        e = &m1;
        cout << "Salary: " << e->Payment() << endl;
        e1.prt();
        m1.prt();
}
```

Compile the project and run it, observing what is printed on the console. Next, add the word virtual before the function **Payment()** in the base class and rerun the program. What happens this time? Why?

1.2. Exercise 2:

For this exercise you must create another project, which again shows the class hierarchy and a new declaration of the base class **Employee**, which will be used as a starting point:

```
#include <iostream>
using namespace std;

class Employee{
    char name[20];
public:
    //constructors
    // virtual function
    virtual void show_info (){
        cout << "Employee" << endl;
    }
    };
```

The steps to be taken are:

1. Enter in the ***base*** class the ***member variable*** that contains the employee's name. This variable will be ***private*** so that derived classes cannot access that value.

   ```
   char name[20];
   ```

2. Create the ***constructors*** needed to be able to indicate the name. The constructors of the base class are the only ones that, using the ***strcpy()*** function, will assign its value to the name variable. The remaining constructors must call the respective constructor of their base class.
3. Add a ***public member function*** to the ***base class*** (Employee) so that it returns the name of the employee:

   ```
   char *getName() { return name; }
   ```

4. Build all classes so that they only have to assign the name to the inherited variable. So far they won't have other variables.
5. Redefine ***show_info()*** function in all other derived classes so it writes the name of the employee. You must use the ***getName()*** function to get the name, since the variable is private.
6. Add the following code to the **main** function:

   ```
   Employee  Rafa((char*)"Rafa");
   Manager Mario((char*)"Mario");
   Worker  Anton((char*)"Anton");
   Officer   Luis((char*)"Luis");
   Technician   Pablo((char*)"Pablo");
   ```

```
// The type of object pointed by a pointer to the base class
// determines the function that is being called
Employee *pe;
cout << "\nInheritance and Polymorphism:\n" << endl;

pe = &Rafa;     pe->show_info();
pe = &Mario;    pe-> show_info();
pe = &Anton;    pe-> show_info();
pe = &Luis;     pe-> show_info();
pe = &Pablo;    pe-> show_info();
cout << I finished…"  << endl;
```

7. Now compile and run the program. Watch what happens. Then delete the word ***virtual*** in the declaration of the ***show_info()*** function in the base class ***Employee***, observing the difference in the output of the program. This is a clear example of ***polymorphism***. Once understood that difference include back the word ***virtual*** and go to next exercise.

1.3. Exercise 3:

The objective of this exercise is to add a member variable, in order to distinguish the management of ***private*** member variables with regard to ***protected***. In addition, you must create a function that displays all the information available about an employee. For that, you must start from the previous exercise and modify it appropriately. To avoid copying all the files above, we will continue with the same project.

The changes to be made are:

1. Enter a ***member variable*** in the ***base*** class that contains the employee's ***salary***. This variable will be ***private*** so that derived classes can not access its value.

   ```
   long salary;
   ```

2. Add member variable ***degree*** to the class ***Manager***. This variable is a pointer to a string of characters (using dynamic memory allocation), and it can be defined as ***protected***.

   ```
   char *degree;
   ```

3. Add the member variable ***position*** to the class ***Worker***. This variable is a pointer to a string of characters (using dynamic memory allocation) and it can be defined as ***protected*** so that classes derived from the class ***Worker*** can access it.

   ```
   char *position;
   ```

4. Create the necessary ***constructors*** to be able to indicate the salary, the degree (in case it is a manager) or the position (in the case of a worker).

5. Create the ***public*** member function ***GetSalary()*** in the base class ***Employee***, which returns the salary of each employee.

   ```
   long GetSalary();
   ```

6. The ***virtual*** function ***show_info()*** must display on the screen all the information stored for each employee: name, salary, and also their position or degree, depending on whether it is a worker or a manager.

7. In the ***main()*** program an array of 6 pointers to employees is going to be created. Previously, the number of employees will be defined with a ***define***:

   ```
   #define NUM_EMPLOYEES 6
   ```

8. Then, in the same file, you must create the employees by calling to each of the constructors.

```
Employee* EmployeeList[NUM_EMPLOYEES];

EmployeeList[0] = new Manager((char*)"Carla", 35000,(char*)"Economist");
EmployeeList[1] = new Manager ((char*)"Juan", 38000,(char*)"Engineer");
EmployeeList[2] = new Officer((char*)"Pedro", 18000, (char*)" Officer 1");
EmployeeList[3] = new Officer ((char*)"Luisa", 15000, (char*)"Officer 2");
EmployeeList[4] = new Technician((char*)"Javier", 19500, (char*)"Welder");
EmployeeList[5]     =     new     Technician     ((char*)"Amaia",     12000,
(char*)"Electrician");
```

9. Finally, after creating the objects, the ***show_info()*** function of each employee should be called, to show their information on screen.

```
for(int i=0;i<NUM_ EMPLOYEES;i++)
    EmployeeList[i]->show_info();
```

1.4. Exercise 4:

Add the class given below. This class is intended to group the name of a task and the performance level of the task.

```
class task
{
public:
        char *name;
        double level;
public:
        task() {}
        ~task() {}

};
```

1. Include within the Worker class, a dynamic array called "***list**" of task objects as a protected variable, and include an integer variable **"cont"** that will be used as a counter for the number of tasks added (see below). Note that you have to do the dynamic memory allocation of the variable **"*list"** of objects of type task inside the constructor of the Worker class (give it a dimension of 20). Don't forget to put the delete statement inside the destructor.

```
class Worker : public Employee
{
protected:
        task* list;
        int cont;
protected:
        char* position;

public:

        //functions
}
```

2. Create afunction **add_task()** that allows a user to introduce a task for employees of type Worker (Officer and Technichian).

```
EmployeeList[2]->add_task((char*)"Redactar", 8);
EmployeeList[2]->add_task ((char*)"Procesar", 6);
EmployeeList[4]->add_task ((char*)"Soldar", 10);
EmployeeList[4]->add_task((char*)"Relleno", 7);
```

3. Modify the **show_info()** function so that it also displays all tasks for each Worker.
4. Change the **"*list"** variable from protected to private. In this case you cannot access the variables directly from the derived classes, so you will have to do it through a **prt_task()**

function. Does it need to be defined in the employee class, or is it sufficient to be defined only in the worker class?

5. How would the code change if the variables **name** and **level** of the task class were private?

1.5. Exercise 5:

After testing the correct operation of the previous exercise, you have to modify the program to ask the user the number of employees to be created and to request all data from the newly created object (***name, salary, position or degree***, etc. depending on the type of object to be created).

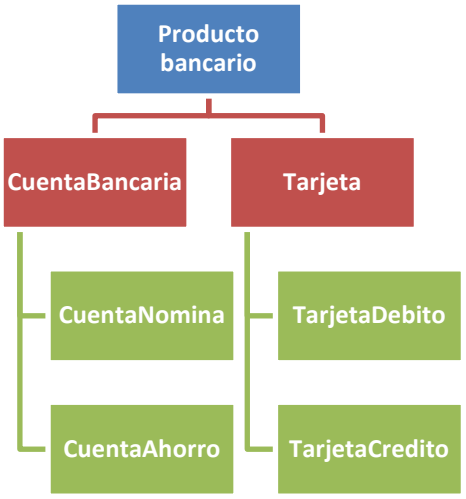
You must make the ***main*** program to have a menu with the following options:

1. Choose what type of employee to create: Manager, Officer or Technician.
2. Make a list of all the employees introduced.
3. Search for a specific employee by name, and
4. Exit from the program.

Remember that all objects should be stored in the list ***EmployeeList*** and it must have a unique variable ***counter*** to keep track of the objects created.

1.6. Ejercicio de examen: Gestion de productos bancarios con herencia

En este ejercicio se pretende simular la programación de algunas funciones que permitan la gestión de productos bancarios. Para ello se considera la siguiente jerarquía de clases, que se muestra en la figura que aparece a la derecha.



La clase `BankProduct` tiene una única variable privada de tipo string, llamada ***holder*** (titular).

La clase `BankAccount` tiene como variable pública un vector de double llamado ***balance***, y como variable protegida un vector de string llamado ***type***. El primero almacenará todos los movimientos que se realicen sobre la cuenta (transferencias, pagos…). El segundo almacenará en una cadena de caracteres el tipo de movimiento del que se trata (posteriormente se muestra un ejemplo).

La clase `SalaryAccount` (Cuenta Nómina) y la clase `SavingsAccount` (Cuenta Ahorro), heredadas de la clase `BankAccount`, no tienen variables propias. La particularidad de la clase `SalaryAccount` es que permite realizar operaciones de pago y transferencias, mientras que la clase `SavingsAccount` solo permite transferencias.

La clase `Card` tiene como única variable un puntero a un objeto de tipo `SalaryAccount`, llamado ***associated_account***.

La clase `DebitCard`, heredada de la clase `Card`, no tiene variables propias. La clase `CreditCard`, heredada de la clase `Card`, tiene una variable privada de tipo double llamada ***credit***. Esta

variable va acumulando el valor de los pagos ejecutados mediante esta tarjeta.

```
class BankProduct {
private:
   string holder;
public:
   BankProduct(string _holder)
   // Add more functions if necessary
};
```

```
class BankAccount : public BankProduct {
public:
   vector<double> balance;
protected:
   vector<string> type;
public:
   BankAccount();
   BankAccount(string _holder, double _balance);
   // Add more functions if necessary
};

class SalaryAccount : public BankAccount {
public:
   SalaryAccount();
   SalaryAccount(string _holder, double _balance);
   // Add more functions if necessary
};

class SavingsAccount : public BankAccount {
    double interest;
public:
   SavingsAccount();
   SavingsAccount(string _holder, double _balance);
   // Add more functions if necessary
};
```

```
class Card : public BankProduct {
protected:
   SalaryAccount * associated_account
public:
   Card();
   Card(SalaryAccount* _associated_account);
   Card(string _holder,  SalaryAccount* _associated_account);
   // Add more functions if necessary
};
class DebitCard : public Card {
public:
   DebitCard();
   DebitCard(string _holder, SalaryAccount* _associated_account);
   DebitCard(SalaryAccount* _associated_account);
   // Add more functions if necessary
};

class CreditCard : public Card {
private:
   double credit;
public:
   CreditCard();
   CreditCard(SalaryAccount* _associated_account);
   CreditCard(string _holder, SalaryAccount* _associated_account);
   // Add more functions if necessary
};
```

1. Constructores, destructores, asignación e inicialización de variables. Se le proporciona el main con el que se comprobará la programación de las funciones a continuación.

    a. Los constructores de las clases `BankAccount` y sus derivadas deberán asignar el nombre del ***holder*** y el primer valor del vector balance y el vector ***type***. El primer valor del vector ***type*** será "Initial".

    b. Cuando se ejecuten los constructores por defecto de `BankAccount` y sus derivadas, el programa deberá pedir obligatoriamente el nombre del ***holder*** y asignar al primer valor del vector ***balance*** un cero.

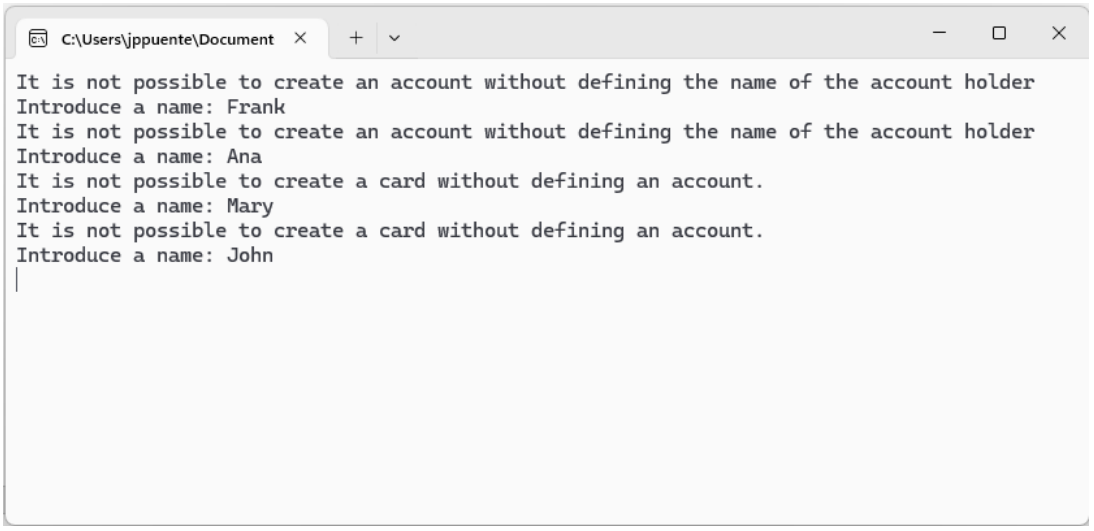
    c. Los constructores de la clase `Card` y sus derivadas deberán permitir: introducir sólo la cuenta asociada, en cuyo caso el ***holder*** de la tarjeta será el mismo que el de la cuenta; o bien introducir el ***holder*** de la tarjeta y la cuenta a la que está asociada, ya que una cuenta puede tener varias tarjetas asociadas con distintos ***holder***.

    d. Cuando se ejecuten los constructores por defecto de `Card` y sus derivadas, el programa deberá pedir obligatoriamente el nombre del ***holder*** y crear una `SalaryAccount` con el mismo ***holder*** y asignar al primer valor del vector ***balance*** un cero. En el caso de la `CreditCard` también inicializará el valor de la variable ***credit*** a cero.

    e. A la hora de crear los destructores, tenga en cuenta que, al destruir una tarjeta, no se debe destruir la cuenta asociada, ya que esta puede seguir existiendo, aunque no haya tarjetas asociadas.

    f. Finalmente deberá almacenar todas las cuentas y tarjetas en las variables ***list_cn***, ***list_ca*** y ***list_t***. Los objetos `SalaryAccount` asociados a ***cn3*** y ***ca3*** también deben guardarse en ***list_cn***.

```
void main()
{
   SalaryAccount *cn1, *cn2, *cn3;
   cn1 = new SalaryAccount("Andrea",1700);
   cn2 = new SalaryAccount("Peter", 1500);
   cn3 = new SalaryAccount();

   SavingsAccount* ca1, *ca2, *ca3;
   ca1 = new SavingsAccount("Diana", 2000);
   ca2 = new SavingsAccount("Paul", 2000);
   ca3 = new SavingsAccount();

   DebitCard* td1, * td2, *td3;
   CreditCard *tc1, * tc2, *tc3;
   // Same holder on the Card and on the account
   td1 = new DebitCard(cn1);
   tc1 = new CreditCard(cn1);

   // Different holder in the Card and in the account
   td2 = new DebitCard("Marc", cn1);
   tc2 = new CreditCard("Esther", cn2);

   // Same holder on the Card and on the account using default constructor
   td3 = new DebitCard();
   tc3 = new CreditCard();

   // Save all the created variables within next variables
   vector<SalaryAccount*> list_cn;
   SavingsAccount** list_ca;
   vector<Card*> list_t;

   // main continues below
```



2. Mostrar datos almacenados por pantalla. Se le proporciona al final del apartado parte del código *main* y la salida esperada por consola.

    a. Muestre por pantalla los datos de todas las cuentas creadas. Deberá mostrar la dirección en la que se encuentra almacenada cada cuenta, así como el ***holder*** y el último valor almacenado en el vector ***balance*** (que se corresponde con el saldo actual). Para ello deberá programar una función llamada ***prt().***

    b. Muestra las tarjetas por tipo, primero las `DebitCard` y luego las `CreditCard`. El nombre de las funciones para imprimir por consola los datos debe ser ***prt()***. En esta sección deberá mostrar la dirección de memoria en la que se encuentra almacenada la tarjeta. Puede llamar a la función ***prt()*** definida en las clases `BankAccount` y derivadas.

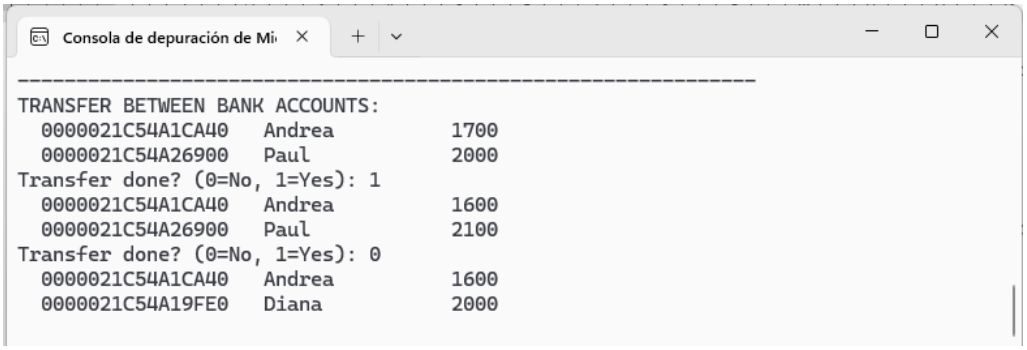
    c. Muestre todas las tarjetas cuyo ***holder*** sea el mismo que el de la cuenta a la que está asociada la tarjeta.

```
// Main continuation
cout<<endl<<"------------------------------------------------------------"<<endl;
cout << "ACCOUNT LIST: " << endl;
cout.setf(ios::left);
cout << "Salary accounts " << endl;
cout << setw(21)<< "  Account adress" << setw(16) << "Account holder" <<
setw(18) << "Account balance" << endl;
for (int i = 0; i < list_cn.size(); i++)
{
list_cn[i]->prt();   // Print the SalaryAccount list
}
cout << "Savings accounts " << endl;
cout << setw(21) << "  Account adress" << setw(16) << "Account holder" <<
setw(18) << "Account balance" << endl;
for (int i = 0; i < 3; i++)
{
list_ca[i]->prt();   // Print the SavingsAccount list
}

cout<<endl<<"------------------------------------------------------------"<<endl;
cout << "CARD LIST BY TYPE: " << endl;
// Print debit cards
cout << "Debit cards" << endl;
```

```
    cout << setw(24) << "  Card adress" << setw(10) << "Type" << setw(15) << "Card
    holder" << setw(21) << "  Account adress" << setw(16) << "Account holder" <<
    setw(18) << "Account balance" << endl;

    // Write your code

    // Print credit cards
    cout << "Credit cards" << endl;
    cout << setw(24) << "  Card adress" << setw(10) << "Type" << setw(15) << "Card
    holder" << setw(21) << "  Account adress" << setw(16) << "Account holder" <<
    setw(18) << "Account balance" << endl;

    // Write your code


    cout<<endl<<"-------------------------------------------------------------"<<endl;
    cout << "LIST OF CARDS WITH THE SAME HOLDER AS THE ACCOUNT HOLDER: " << endl;
    cout << setw(24) << "  Card adress" << setw(10) << "Type" << setw(15) << "Card
    holder" << setw(21) << "  Account adress" << setw(16) << "Account holder" <<
    setw(18) << "Account balance" << endl;

    // Write your code

    // Continued below
```

3. Crea una función llamada ***Transfer*** que realice una transferencia de dinero entre dos cuentas.
    a. Esta función debe devolver una variable booleana que determine si ha sido posible o no realizar la operación. Las variables de entrada son la cuenta de destino y la cantidad que se desea transferir.
    b. La operación solo es posible si el saldo de la cuenta de origen (la cuenta que llama a la función) es mayor que la cantidad que se desea transferir. Deberá comprobarse que dicha cantidad es mayor que cero.
    c. En caso de poder realizarse deberá descontarse dicha cantidad de la cuenta de origen y añadirlo en la cuenta de destino. Este movimiento quedará registrado en el vector ***balance*** y en el vector ***type*** (guardado como "Transferencia") de ambas cuentas.

```
    // Main continuation
    cout<<endl<<"-------------------------------------------------------------"<<endl;
    cout << "TRANSFER BETWEEN BANK ACCOUNTS: " << endl;
    bool b;

    list_cn[0]->prt();
    list_ca[1]->prt();

    b = list_cn[0]->Transferencia(list_ca[1], 100);
    cout << "Transfer done? (0=No, 1=Yes): " << b << endl;
    list_cn[0]->prt();
    list_ca[1]->prt();

    b = list_cn[0]->Transferencia(list_ca[1], 10000);
    cout << "Transfer done? (0=No, 1=Yes): " << b << endl;
    list_cn[0]->prt();
    list_ca[0]->prt();

    // Continued below
```



4. Crea una función llamada ***Payment*** que permita realizar un pago desde una tarjeta (se proporciona el *main* y la salida esperada por consola al final del ejercicio).
    a. Esta función debe devolver una variable booleana que determine si ha sido posible o no realizar la operación. Como variable de entrada tiene un double que se corresponde con la cantidad del pago que se debe realizar.
    b. En el caso de que un objeto `DebitCard` llame a la función:
        i. La operación solo es posible si el saldo de la cuenta de origen (cuenta a la que está asociada la tarjeta que llama a la función) es mayor que la cantidad que se desea pagar. Deberá comprobarse que dicha cantidad es mayor que cero.
        ii. En caso de poder realizarse deberá descontarse dicha cantidad de la cuenta asociada a la tarjeta, almacenando el nuevo valor del saldo en el vector ***balance***. Este movimiento quedará registrado en el vector ***type*** como "Debit card payment".
    c. En el caso de que un objeto `CreditCard` llame a la función, el valor del pago se sumará a la variable ***credit***, pero no se cargará en la cuenta y por tanto tampoco quedará registrado en el vector ***balance*** y en el vector ***type***.

5. Crea una función llamada ***CreditLiquidation*** que permita cargar los gastos almacenados en la variable credit de la tarjeta en la cuenta asociada (se proporciona el *main* y la salida esperada por consola al final del ejercicio).
    a. Esta función debe devolver una variable booleana que determine si ha sido posible o no realizar la operación. No tiene variables de entrada.
    b. La operación solo es posible si el saldo de la cuenta de origen (cuenta a la que está asociada la tarjeta que llama a la función) es mayor que la cantidad registrada en la variable crédito.
    c. En caso de poder realizarse deberá descontarse dicha cantidad de la cuenta asociada a la tarjeta, almacenando el nuevo valor del saldo en el vector ***balance***. Este movimiento quedará registrado en el vector ***type*** como "Credit liquidation".
6. Crea una función llamada ***Movements()*** que muestre por consola todos los movimientos registrados en una cuenta.
    a. Esta función no devuelve variables y no tiene variables de entrada.
    b. Deberá mostrar el tipo de movimiento, la cantidad que se ha movido en cada

operación y el saldo tras cada operación (ver ejemplo en pantallazo de salida esperada por consola).

```
    // Main continuation
    cout<<endl<<"-----------------------------------------------------------"<<endl;
    cout << "PAYMENTS WITH CARDS: " << endl;
    b = list_t[0]->Payment(25);
    cout << "Paymentment done? (0=No, 1=Yes): " << b << endl;
    list_cn[0]->prt();

    b = list_t[0]->Payment(20000);
    cout << "Paymentment done? (0=No, 1=Yes): " << b << endl;
    list_cn[0]->prt();

    b = list_t[1]->Payment(3);
    cout << "Paymentment done? (0=No, 1=Yes): " << b << endl;
    list_cn[0]->prt();

    b = list_t[1]->Payment(7);
    cout << "Paymentment done? (0=No, 1=Yes): " << b << endl;
    list_cn[0]->prt();

    b = list_t[1]->Payment(8);
    cout << "Paymentment done? (0=No, 1=Yes): " << b << endl;
    list_cn[0]->prt();

    cout << endl;
    b = list_t[1]->CreditLiquidation();
    cout << "Credit card liquidation? (0=No, 1=Yes)" << b << endl;
    list_cn[0]->prt();

    cout<<endl<<"-----------------------------------------------------------"<<endl;
    cout << "ACCOUNT MOVEMENTS: " << endl;
    list_cn[0]->movements();
    cout << endl;
    list_ca[1]->movements();
}
```

```
Consola de depuración de Mi

-----------------------------------------------------------------
ACCOUNT LIST:
Salary accounts
  Account adress     Account holder  Account balance
  000001DAA881CA40   Andrea          1700
  000001DAA881CD10   Peter           1500
  000001DAA881C7D0   Frank           0
  000001DAA8828BD0   Frank           0
  000001DAA8828D10   John            0
Savings accounts
  Account adress     Account holder  Account balance
  000001DAA8819FE0   Diana           2000
  000001DAA8826900   Paul            2000
  000001DAA8828890   Ana             0

-----------------------------------------------------------------
CARD LIST BY TYPE:
Debit cards
  Card adress          Type     Card holder     Account adress     Account holder  Account balance
  000001DAA8828950     Debit    Andrea    -->   000001DAA881CA40   Andrea          1700
  000001DAA8828A50     Debit    Marc      -->   000001DAA881CA40   Andrea          1700
  000001DAA8828B50     Debit    Frank     -->   000001DAA8828BD0   Frank           0
Credit cards
  Card adress          Type     Card holder     Account adress     Account holder  Account balance
  000001DAA88289D0     Credit   Andrea    -->   000001DAA881CA40   Andrea          1700
  000001DAA8828AD0     Credit   Esther    -->   000001DAA881CD10   Peter           1500
  000001DAA8828C90     Credit   John      -->   000001DAA8828D10   John            0

-----------------------------------------------------------------
LIST OF CARDS WITH THE SAME HOLDER AS THE ACCOUNT HOLDER:
  Card adress          Type     Card holder     Account adress     Account holder  Account balance
  000001DAA8828950     Debit    Andrea    -->   000001DAA881CA40   Andrea          1700
  000001DAA88289D0     Credit   Andrea    -->   000001DAA881CA40   Andrea          1700
  000001DAA8828B50     Debit    Frank     -->   000001DAA8828BD0   Frank           0
  000001DAA8828C90     Credit   John      -->   000001DAA8828D10   John            0
```